

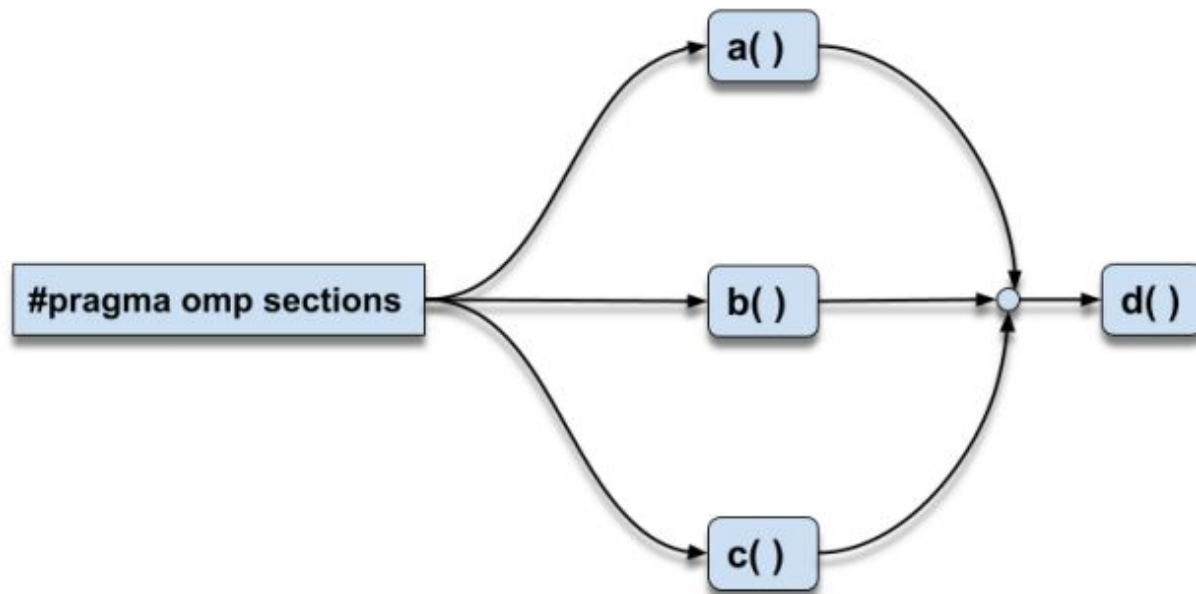
IMD1116
COMPUTAÇÃO DE
ALTO DESEMPENHO
MEMÓRIA COMPARTILHADA
OPENMP - DIVISÃO DE TRABALHO

Prof. Samuel Xavier de Souza
Prof. Carla Santana

PROGRAMAÇÃO EM MEMÓRIA COMPARTILHADA

- Objetivos deste tópico
 - #pragma omp sections
 - Cláusula schedule
 - static
 - dynamic
 - guided
 - chunksize

Diretiva sections



```
#pragma omp sections [ cláusulas ]
{
    # pragma omp section
    {
        a ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        b ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        c ( ) ; // bloco estruturado
    }
}
d();
```

Diretiva sections

- A diretiva sections distribui o trabalho entre diversas threads limitadas a uma região paralela definida.
- Essa diretiva permite a execução em paralelo de blocos separados de código (por exemplo, diversas subrotinas independentes)
- Não escalável bem, porque a quantidade de paralelismo disponível é determinada no código-fonte, sendo portanto pouco utilizada.
- A diretiva **sections** indica o início de uma construção, que contém diversas diretivas **section**.

```
#pragma omp sections [ cláusulas ]
{
    # pragma omp section
    {
        a ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        b ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        c ( ) ; // bloco estruturado
    }
}
d( );
```

Diretiva sections

- Algumas regras para a utilização dessas diretivas são:
 - Uma diretiva **section** não pode aparecer fora do bloco estruturado abrangido pela diretiva **sections**;
 - Cada bloco estruturado, guardado por uma diretiva **section**, será executado em paralelo por uma thread distinta;
 - Existe uma barreira implícita definida no final da região paralela associada à diretiva **sections**, se não houver uma cláusula **nowait** especificada

```
#pragma omp sections [ cláusulas ]
{
    # pragma omp section
    {
        a ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        b ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        c ( ) ; // bloco estruturado
    }
}
d( );
```

Diretiva parallel sections



```
#pragma omp parallel sections [ cláusulas ]
{
    #pragma omp section
    {
        // bloco estruturado
    }
    #pragma omp section
    {
        // bloco estruturado
    }
}
```

Atividade Extra

Considere uma função contínua $f(x)$ definida em um intervalo $[a,b]$. Você deverá implementar dois cálculos independentes:

- Aproximação da integral definida de $f(x)$ usando o método do trapézio
- Aproximação da derivada de $f(x)$ em pontos discretos usando diferenças finitas

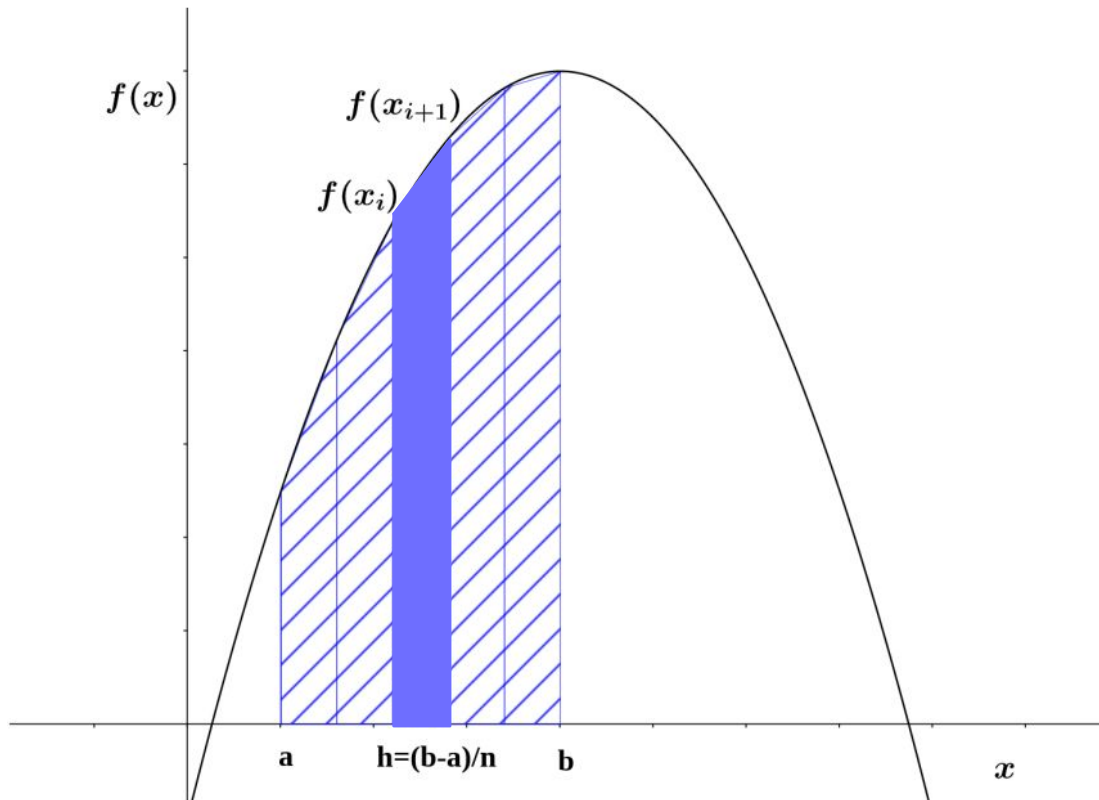
Implemente um programa em C utilizando OpenMP que:

1. Defina uma função $f(x)$ (ex: $\sin(x)$, x^2 ...)
2. Discretize o intervalo $[a,b]$ em n pontos
3. `#pragma omp parallel sections` para executar em paralelo:
 - a. uma seção responsável pelo cálculo da integral
 - b. outra seção responsável pelo cálculo da derivada

Atividade Extra

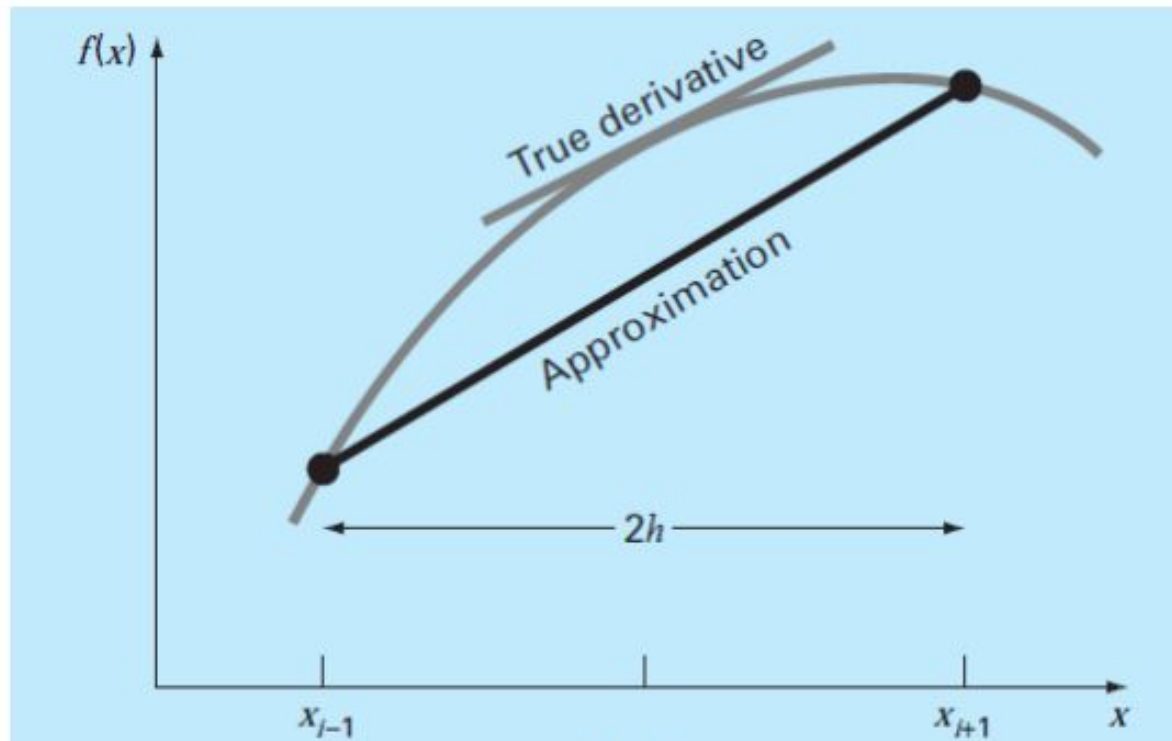
1. Execute o programa variando:
 - a. número de pontos n (tamanho do dx)
 - b. número de threads (OMP_NUM_THREADS)
2. Analise:
 - a. tempo de execução
 - b. possível ganho de desempenho

Método do Trapézio



$$h * \left[\frac{f(x_o)}{2} + \frac{f(x_n)}{2} + \sum_{i=1}^{n-1} f(x_i) \right]$$

Diferenças Finitas



$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{2h}$$

Cláusula schedule

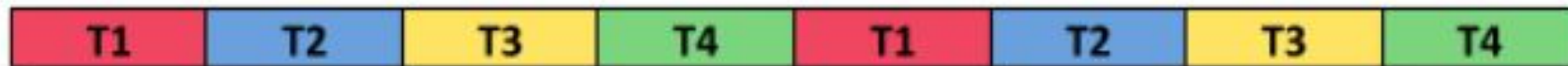
- A cláusula schedule permite uma variedade de opções para especificar quais threads executarão as iterações dos laços.

schedule (tipo[, chunk_size])

- Tipo:
 - static
 - dynamic
 - guided
 - auto
 - runtime
- **chunk_size** é uma expressão inteira com valor positivo.

Cláusula schedule - static

- No escalonamento **static**,
- Sem a especificação de **chunk_size**:
 - O espaço de iteração é dividido em pedaços (aproximadamente) iguais e cada pedaço é atribuído a cada thread (escalonamento em bloco).
- Se o valor de **chunk_size** for especificado:
 - O espaço de iteração é dividido em pedaços, cada um com **chunk_size** iterações, e os pedaços são atribuídos ciclicamente a cada thread (escalonamento cíclico por bloco).



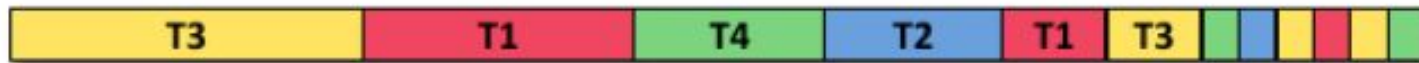
Cláusula schedule - dynamic

- O escalonamento dynamic divide o espaço de iteração em pedaços com **chunk_size** iterações, e os atribui para as threads com uma política “**primeiro a chegar, primeiro a ser atendido**”, isto é, se uma thread terminou um pedaço, ela receberá o próximo pedaço da lista.
- Quando nenhum valor de **chunk_size** for especificado, o valor padrão assumido é 1



Cláusula schedule - guided

- O escalonamento guided é similar ao dynamic, **mas os pedaços iniciam grandes e se tornam menores exponencialmente.**
- O tamanho do próximo pedaço é (com arredondamento para cima) **o número de iterações restantes dividido pelo número de threads** .
- O valor **chunk_size** especifica o tamanho mínimo dos pedaços
 - exceto para o pedaço com as últimas iterações, que pode ser menor .
- Quando nenhum valor de **chunk_size** for especificado, o padrão é igual a 1



```
#pragma omp parallel for schedule(guided, 4)
for (int i = 0; i < 100; i++) {
    trabalho(i);
}
// início:   blocos grandes → 25, 18, 14...
// meio:     blocos médios → 10, 8...
// final:    blocos pequenos → 4, 4, 4...
```

Cláusula schedule - auto

- O escalonamento auto dá liberdade para o ambiente de execução escolher qual método de escalonamento será utilizado.
- Isso pode ser útil, por exemplo, quando **o laço paralelo é executado diversas vezes**, permitindo ao ambiente de execução convergir para um tipo de escalonamento com um bom balanceamento de carga e com baixa sobrecarga.

Cláusula schedule - runtime

- O escalonamento runtime **delega a escolha do escalonamento para o momento da execução do programa**
- Podendo ser determinado pelo valor da variável de ambiente **OMP_SCHEDULE** ou pela chamada à função **omp_set_schedule()**.
- Quando presente, a variável de ambiente OMP_SCHEDULE especifica o tipo de escalonamento a ser utilizado nos laços.
- Esses tipos podem ser somente **static**, **dynamic**, **guided** ou **auto**.



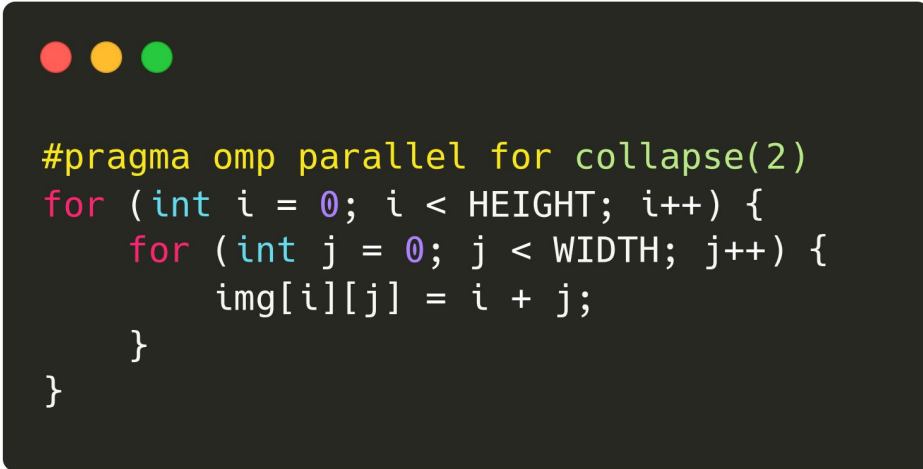
```
export OMP_SCHEDULE = tipo [, tam ]  
export OMP_SCHEDULE = "guided ,4"  
export OMP_SCHEDULE = "static ,1000"
```

Cláusula schedule

- Qual tipo de escalonamento deve ser utilizado?
 - **static**: melhor para laços balanceados – menor sobrecarga;
 - **static, n**: melhor para laços com desbalanceamento suave;
 - **dynamic**: útil se as iterações têm grande variação de carga, mas acaba com a localidade espacial dos dados e tem o custo adicional de troca de threads;
 - **guided**: frequentemente menos cara que dynamic, mas tenha cuidado com laços onde as primeiras iterações são as mais demoradas;
 - **auto**: pode ser útil se o laço é executado diversas vezes ao longo da execução do programa;
 - Use **runtime** para uma experimentação adequada.

Cláusula collapse

- A cláusula collapse no OpenMP serve para **combinar (colapsar) múltiplos loops** aninhados em um único espaço de iteração, permitindo uma melhor distribuição de trabalho entre as threads.
- A cláusula collapse é utilizada para aumentar o número total de iterações que serão particionadas entre as threads disponíveis para execução
- Aumenta a granularidade do trabalho a ser realizado por cada thread.
- Se a quantidade de trabalho a ser realizado por cada thread for significativa (após a aplicação da compactação), isso poderá melhorar a escalabilidade paralela programa.



```
#pragma omp parallel for collapse(2)
for (int i = 0; i < HEIGHT; i++) {
    for (int j = 0; j < WIDTH; j++) {
        img[i][j] = i + j;
    }
}
```

PARTICIONAMENTO DE DADOS E BALANCEAMENTO DE CARGA

Tarefa 11:

Escreva um código que simule o movimento de um fluido ao longo do tempo usando a equação de Navier-Stokes, considerando apenas os efeitos da viscosidade.

Desconsidere a pressão e quaisquer forças externas. Utilize **diferenças finitas** para discretizar o espaço e simule a evolução da velocidade do fluido no tempo. Inicialize o fluido parado ou com velocidade constante e verifique se o campo permanece estável. Em seguida, crie uma pequena perturbação e observe se ela se difunde suavemente. Após validar o código, paralelize-o com OpenMP e explore o impacto das **cláusulas schedule e collapse** no desempenho da execução paralela.

- A equação de Navier-Stokes
 - Descreve como um fluido se move, acelera e se deforma por causa da pressão, da viscosidade e de forças externas.
- Aplicações práticas
 - Movimento de ar (vento, turbulência atmosférica).
 - Fluxo de água (rios, oceanos, tubulações).
 - Aerodinâmica (aviões, carros).
 - Medicina (fluxo sanguíneo).
 - Climatologia (modelagem do clima).
 - Engenharia (motores, bombas, ventilação).

Referências

